Keyed Improvement

Source Updates, Path Semantics and Lattice Closure

Mettapedia Working Notes

31 October 2026

Abstract

Seven Lean modules connect a source-shaped remove/add improvement to keyed minimum replacement, and an improvement-driven shortest-path worklist to an independent weighted-path semantics. The worklist terminates over a finite vertex universe with nonnegative natural costs. Its final labels equal the least closure in the reverse numerical order, using the existing chaotic iteration theory. Source-shaped adjacency loops admit a stronger trace law: their once-read source distance can be retained across their own updates. Final-distance confluence is kept separate from propagation counts, source round counters, concurrent publication and native implementation costs.

1 Physical deletion can denote monotone information improvement

The inspected shortest-path program keeps at most one distance row per key. Its `improve` operation inserts an absent distance, or removes the old row and inserts a strictly smaller distance. A successful improvement returns the key; otherwise it returns no answer. Its surrounding driver builds a new frontier and ultimately returns a round counter as well as exposing distances.

`KeyedImprovement` models a slot as `OptionValue` and independently models its finite row relation. Encoding a slot store yields one row per occupied key. The proved row law preserves the entire encoded relation and the strict-improvement flag after a completed update. It requires the functional-relation invariant and the correct observed old value.

For exact natural distances, absence is genuine ∞. The keyed update is

$$d[k] \leftarrow \min(d[k], c).$$

It is the cap operation in the existing `TropicalAffineSummary` algebra. Thus removing and replacing a physical row can represent a monotone semantic operation. Monotonicity belongs to the information order, not to the literal set of physical rows.

Fixed candidate updates commute on final values. Their changed/not-changed flags do not: starting at 10, applying 7 then 5 changes twice, while applying 5 then 7 changes once. Edge-derived candidates also depend on other live labels; their individual transitions are not asserted to commute.

2 Publication and captured observations

A raw remove/add sequence has an observable intermediate absence. Two threads using the same stale old row can additionally destroy the one-value invariant: after the row $(k, 10)$ becomes $(k, 7)$, erasing the nonexistent $(k, 10)$ and adding $(k, 5)$ leaves both $(k, 7)$ and $(k, 5)$.

`KeyedImprovementTransactions` prepares a key, candidate and observed value. Its logical publication validates the observed value against the current key before applying the prepared branch. A stale ticket returns retry, not "no improvement." A fallback recomputes against the live store, and is proved equal to ordinary live improvement. Unrelated key updates need not force retry.

These are value-level publication laws. A native implementation must supply the corresponding atomicity, failure handling and ownership. If occurrence identity is observable, validation must also retain the required occurrence or generation identity; equality of payload values does not certify pointer lifetime. Existing captured-reader laws remain relevant to readers surviving the physical row replacement.

3 Independent shortest-path meaning

An arc has a source, target and natural weight. Parallel edges and zero weights are permitted. `WeightedPathRelaxation` reuses Mathlib's quiver paths and additive path-weight fold. Its reference meaning is

$$D_s(v) = \inf_{p : s \rightsquigarrow v} \mathrm{weight}(p) \quad \text{in } \mathbb{N} \cup \{\infty\}.$$

This definition does not invoke the relaxation algorithm. The empty family of paths has infimum ∞; no finite sentinel denotes unreachability.

Initialization assigns 0 to the source and ∞ elsewhere. Relaxing $e : u \to v$ computes

$$c = d(u) + w(e), \qquad d(v) \leftarrow \min(d(v), c).$$

Every finite label is proved to have an actual source-path witness. Relaxation preserves that provenance by extending a witnessed path.

Conversely, a store below the initial labels and satisfying every edge inequality $d(v) \leq d(u)+w(e)$ lies below the weight of every path, by path induction. Therefore a closed store with path provenance equals D_s. Neither queue emptiness without coverage nor a small-looking numerical assignment is sufficient on its own.

4 A finite, resumable worklist

`ShortestPathWorklist` retains labels, pending arcs and an ordered productive-event log. It initially schedules outgoing arcs of the root. Processing an arc reads the current labels. A strict improvement schedules the improved vertex's outgoing arcs; a non-improvement consumes only the current duty.

The invariant has four parts:

1. labels remain below the initial labels;
2. finite labels retain path provenance;
3. pending arcs belong to the admitted graph;
4. every violated edge inequality has a pending duty.

Only a decrease at a source coordinate can newly violate an outgoing inequality. That is why rescheduling the updated vertex's outgoing arcs preserves coverage. At quiescence, coverage proves closure and hence exact shortest-path distances.

Queue policies preserve duty support, not necessarily duplicate occurrences. DFS, FIFO and deduplicating FIFO satisfy this condition. Deduplicating pending work under the final-distance observer is different from deduplicating source answers. A broken policy that drops newly generated duties terminates with an incorrect distance and is rejected by the queue law.

Termination uses a lexicographic descent: labels strictly decrease on a productive step; otherwise the pending-list length decreases. Over finitely many vertices, the pointwise order on natural distances with top is well-founded. Hence every finite pending state eventually becomes quiescent. This is an existence-of-finite-completion theorem, not a uniform numeric instruction or time bound. The existing `MachineCore.runSlice` supplies the actual bounded runner and exact split-resumption accounting.

5 The correct lattice direction

Smaller numerical distances express better upper bounds, so the information order is the reverse numerical order. `LatticeRelaxationClosure` embeds edge contributions in the existing complete-lattice chaotic-iteration construction and proves that its least closure equals D_s.

Numerically, D_s is the *greatest* feasible label assignment below the initial labels and satisfying the edge inequalities. Calling it the least numerical solution of those inequalities would be wrong: unjustified smaller labels may satisfy them. Path provenance and the information-order reversal keep the two readings distinct.

Every fair schedule of edge contributions eventually reaches the same distance assignment. The concrete worklist also handles an empty graph, where there is no infinite edge-valued schedule to instantiate. No theorem identifies the schedules' productive-event counts or source round counters.

6 A stronger law inside one adjacency loop

The source-shaped `relax` function reads a vertex distance once and then walks its outgoing edges. A live implementation may instead read that source distance at each edge. `RelaxationAdjacency` proves the two loops equal, including their ordered improvement emissions, for any captured outgoing edge list with nonnegative weights and no external interference inside the loop.

An update to a different target leaves the source coordinate alone. A self-loop cannot improve its own source, because $d(u) + w \geq d(u)$. Consequently the source value stays fixed throughout this loop. This is value-level noninterference despite possible syntactic read/write overlap; requiring every possible write key to differ from every read key would be unnecessarily restrictive here.

The negative control improves the source externally from 10 to 0 before an edge of weight 1. The captured and live calculations then propose 11 and 1. The isolated-loop law therefore cannot be used to erase an admitted external update.

7 Controls, observations and qualification

On the graph $s \to b$ of cost 1, $s \to a$ of cost 10, and $b \to a$ of cost 1, DFS and FIFO both finish at $d(b) = 1, d(a) = 2$. Their chronological improvement logs are respectively

$$[(b, 1), (a, 2)] \quad \text{and} \quad [(b, 1), (a, 10), (a, 2)].$$

Thus final-distance equality does not preserve propagation multiplicity. The inspected source also returns its round counter: a native per-key upsert can retain the existing driver order, but changing that driver requires a separate observation theorem or an explicitly different observer contract.

Additional controls cover empty graphs, duplicate edges, unchanged zero-cost cycles, incomplete prefixes, exact resumption, finite sentinels, stale tickets and graph deletion invalidating path witnesses. A non-strict improvement guard requeues a zero-cost self-loop forever. A negative integer self-cycle has no fixed finite distance. The proof is therefore not a blanket termination claim for arbitrary lattices, signed graphs, floating arithmetic or PLN revision. For probabilistic inference, the appropriate information order must be established; taking the largest scalar truth value is not supplied by these laws.

The seven mathematical modules and joined audit use the required Lean build procedure with strict source checks. Module-owned declarations and primitive axiom dependencies are inventoried; compiler runtime counterparts are listed separately. Native source extraction, atomic implementation, liveness under external scheduling and measured costs remain separate qualification duties. No runtime source is changed by this development.